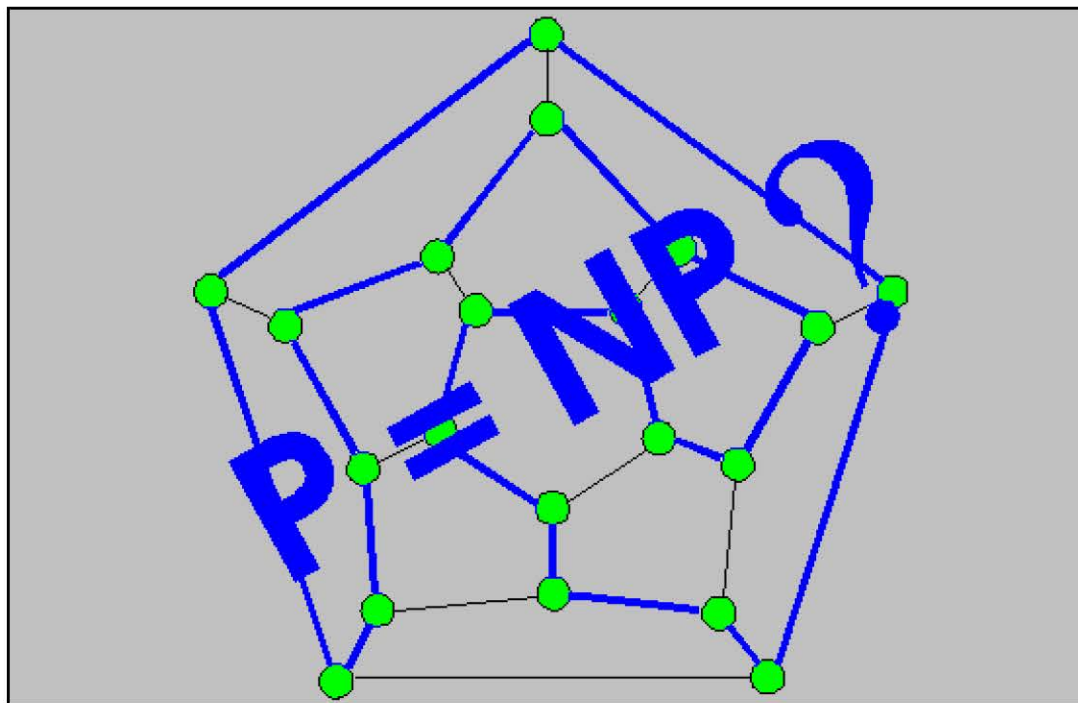




算法设计与分析

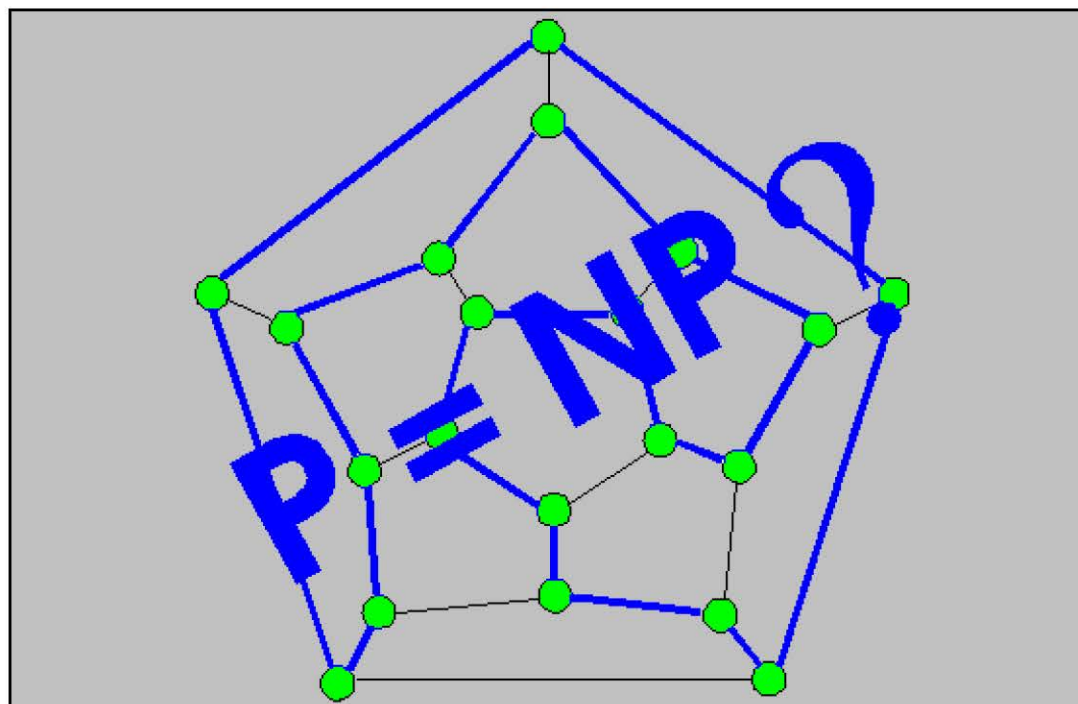
王东滨

dbwang@bupt.edu.cn

教学目标

- 计算机软件的核心是算法，算法描述了计算机软件**解决问题的方法和过程**。算法设计与分析是重要的专业基础课程，是关于算法的方法论。
- 学生通过本课程学习计算机算法设计与分析的主要方法，掌握常用算法设计策略和解决典型问题的具体算法，具备正确分析**算法时间、空间复杂性**的能力，为今后面向实际应用问题，开发有效的计算机算法奠定扎实的基础。

教材： 计算机算法设计与分析（第5版）



王晓东 编著
电子工业出版社

第1章 算法概述

第1章 算法概述

学习要点:

- 理解算法的概念。
- 理解什么是程序，程序与算法的区别和内在联系。
- 掌握算法的计算复杂性概念。
- 掌握算法渐近复杂性的数学表述。
- 掌握用编程语言描述算法的方法。

算法(Algorithm)

算法（**Algorithm**）是指解题方案的准确而完整的描述，是一系列解决问题的清晰指令，算法代表着用系统的方法描述解决问题的策略机制。**能够对一定规范的输入，在有限时间内获得所要求的输出**。如果一个算法有缺陷，或不适合于某个问题，执行这个算法将不会解决这个问题。不同的算法可能用**不同的时间、空间**或效率来完成同样的任务。一个算法的优劣可以用**空间复杂度与时间复杂度**来衡量。

算法(Algorithm)

算法是指解决问题的一种方法或一个过程。

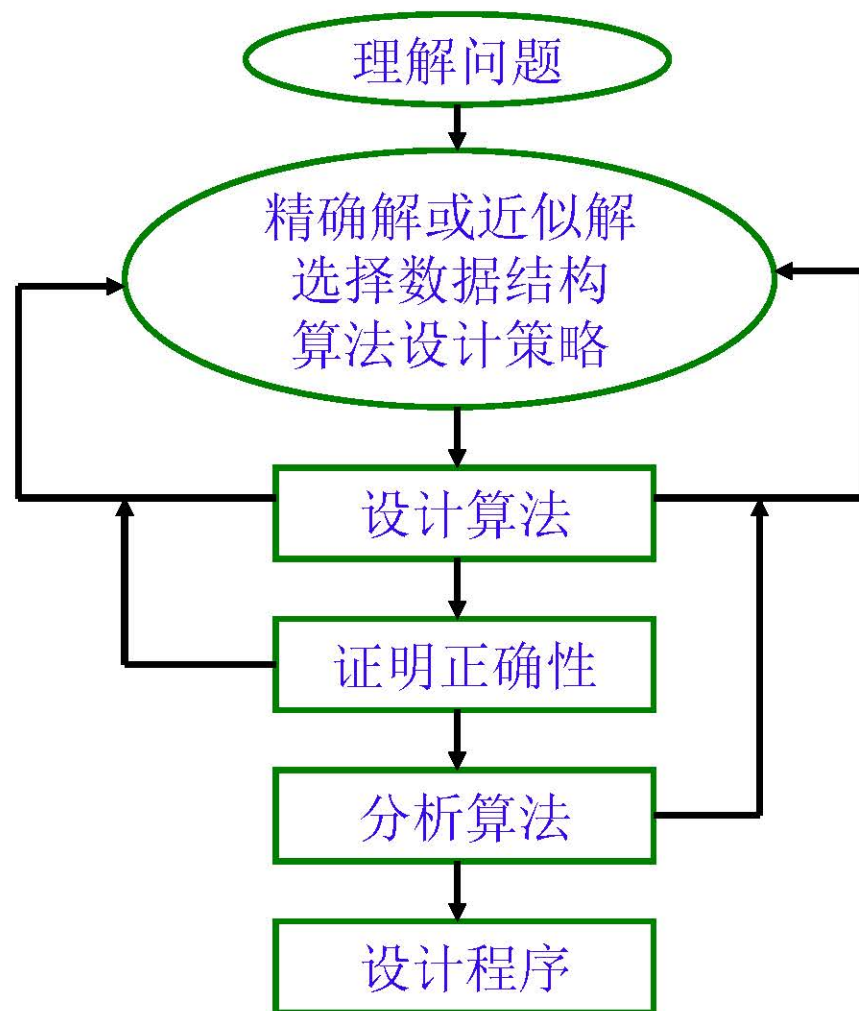
算法是若干指令的有穷序列，满足性质：

- (1)**输入**：有零或若干个外部提供的量作为算法的输入。
- (2)**输出**：算法产生至少一个量作为输出。
- (3)**确定性**：组成算法的每条指令是清晰，无歧义的。
- (4)**有限性**：算法中每条指令的执行次数是有限的，执行每条指令的时间也是有限的。

程序(Program)

- 程序是算法用某种程序设计语言的具体实现。
- **程序可以不满足算法的性质(4)。**
- 例如操作系统，是一个在**无限循环**中执行的程序，因而不是一个算法。
- 操作系统的各种任务可看成是单独的问题，每一个问题由操作系统中的一个子程序通过特定的算法来实现。该子程序得到输出结果后便终止。

问题求解(Problem Solving)



算法复杂性

算法复杂性 = 算法所需要的**计算机资源**

- 算法的**时间复杂性** $T(n)$;
- 算法的**空间复杂性** $S(n)$ 。

其中 n 是问题的规模（输入大小）。

目标:

对于给定问题，设计出复杂性尽可能低的算法。

算法复杂性

$$C=F(N, I, A)$$

C: 算法复杂性

N: 解的问题的规模

I: 算法的输入

A: 算法本身

$F(N, I, A)$: 由参数确定的三元函数

时间复杂性 $T = T(N, I, A)$; **空间复杂性** $S = S(N, I, A)$

通常A隐含在复杂性函数名当中, 简写为:

$$T = T(N, I) ; S = S(N, I)$$

算法的时间复杂性

(1) **最坏情况**下的时间复杂性

$$T_{\max}(n) = \max\{ T(I) \mid \text{size}(I)=n \}$$

(2) **最好情况**下的时间复杂性

$$T_{\min}(n) = \min\{ T(I) \mid \text{size}(I)=n \}$$

(3) **平均情况**下的时间复杂性

$$T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$$

其中 I 是问题的规模为 n 的实例， $p(I)$ 是实例 I 出现的概率。

算法渐近复杂性

- $T(n) \rightarrow \infty$, as $n \rightarrow \infty$;
- $(T(n) - t(n)) / T(n) \rightarrow 0$, as $n \rightarrow \infty$;

$t(n)$ 是 $T(n)$ 的渐近性态，为算法的渐近复杂性。在数学上， $t(n)$ 是 $T(n)$ 的渐近表达式，是 $T(n)$ 略去低阶项留下的主项。

as $n \rightarrow \infty$, $t(n)$ 渐进于 $T(n)$, $t(n)$ 替代 $T(n)$ 作为算法的复杂性度量。

渐近分析的记号

在下面的讨论中，对所有 n ， $f(n) \geq 0$ ， $g(n) \geq 0$ 。

(1) 渐进上界记号 O

$$f(n) = O(g(n))$$

{存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ 有： $0 \leq f(n) \leq cg(n)$ }

(2) 非紧上界记号 o

$$f(n) = o(g(n))$$

{存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ 有： $0 \leq f(n) < cg(n)$ }。

(3) 渐进下界记号 Ω

$$f(n) = \Omega(g(n))$$

{存在正常数 c 和 n_0 使得对所有 $n \geq n_0$ 有: $0 \leq cg(n) \leq f(n)$ }

(4) 非紧下界记号 ω

$$f(n) = \omega(g(n))$$

{存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq cg(n) < f(n)$ }。

$$f(n) \in \omega(g(n)) \Leftrightarrow g(n) \in o(f(n))$$

(5) 紧渐近界记号 Θ

$$f(n) = \Theta(g(n))$$

当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$ ， $f(n)$ 与 $g(n)$ 同阶

{存在正常数 c_1, c_2 和 n_0 使得对所有 $n \geq n_0$ 有： $c_1 g(n) \leq f(n) \leq c_2 g(n)$ }

定理1: $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

证明： $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$

1) 设 $F(n) = O(f(n))$ ，则：

存在正常数 c_1 和 n_1 ，使得对所有 $n \geq n_1$ 有： $0 \leq F(n) \leq c_1 f(n)$

2) 类似地，设 $G(n) = O(g(n))$ ，则：

存在正常数 c_2 和 n_2 ，使得对所有 $n \geq n_2$ 有： $0 \leq G(n) \leq c_2 g(n)$

3) 令 $c_3 = \max\{c_1, c_2\}$ ， $n_3 = \max\{n_1, n_2\}$ ， $h(n) = \max\{f(n), g(n)\}$ ，则：

$$0 \leq F(n) \leq c_1 f(n) \leq c_3 f(n) \leq c_3 h(n)$$

$$0 \leq G(n) \leq c_2 g(n) \leq c_3 g(n) \leq c_3 h(n)$$

$$\text{因而， } O(f(n)) + O(g(n)) = F(n) + G(n) \leq c_3 h(n) + c_3 h(n) = 2c_3 h(n)$$

$$= O(h(n)) = O(\max\{f(n), g(n)\})$$

第1章 小结

重点:

算法复杂度定义、 $O(g(n))$ 、 $\Omega(g(n))$

$$f(n) = O(g(n))$$

{存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$ 有: $0 \leq f(n) \leq cg(n)$ }

难点:

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$$

$$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$

$$O(f(n)) * O(g(n)) = O(f(n) * g(n))$$